Compte rendu – TP 2

Création d'un service web REST avec Spring Boot

Module SOA – 3IDL – Institut Supérieur d'Informatique – 2022/2023

1. Objectif

Ce TP consiste à créer un service web REST en Java avec Spring Boot. Le service gère des employés (création, lecture, mise à jour, suppression) stockés dans une base de données MySQL.

2. Environnement de travail

IntelliJ IDEA, JDK, Spring Boot (Spring Web, Spring Data JPA, MySQL Driver), MySQL 8 et le client HTTP intégré d'IntelliJ pour tester les appels à l'API.

3. Architecture du projet

Le projet **demo** a été généré avec Spring Initializr puis organisé en couches, sous le package **com.example.demo** :

Couche	Classe	Rôle
model	Employee	Entité JPA mappée sur la table *employee*.
repository	EmployeeRepository	Interface étendant JpaRepository : opérations CRUD sur la base.
service	EmployeeService	Logique métier : créer, lister, modifier et supprimer un employé.
controller	EmployeeController	Expose les endpoints REST sous */api/employees*.

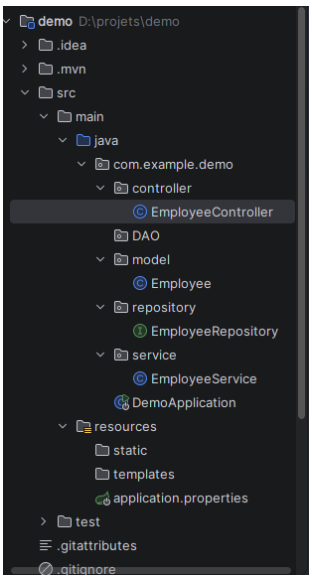
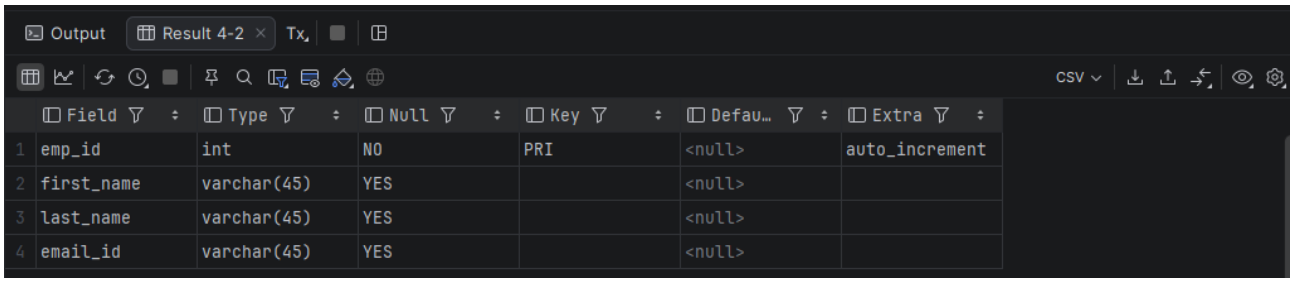


Figure 1 – Structure du projet dans IntelliJ

4. Base de données et configuration

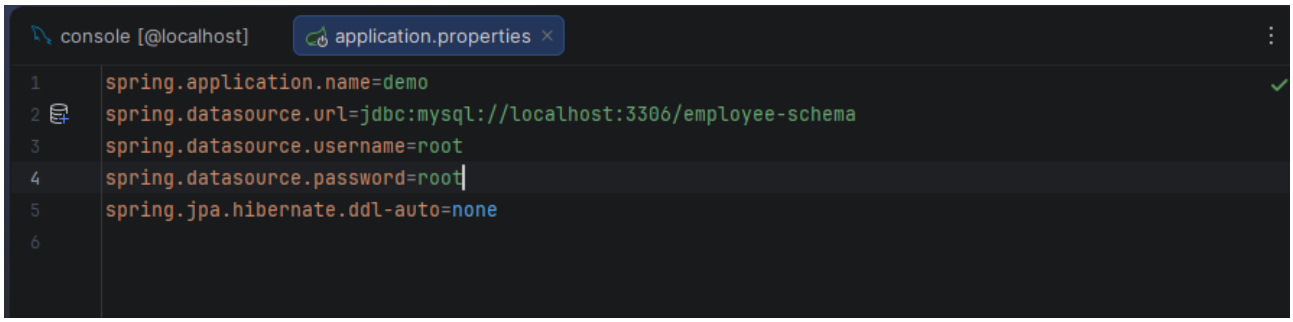
La base **employee-schema** et la table **employee** ont été créées dans MySQL (clé primaire *emp_id* auto-incrémentée, colonnes *first_name*, *last_name* et *email_id*).



	Field	Type	Null	Key	Defau…	Extra
1	emp_id	int	NO	PRI	<null>	auto_increment
2	first_name	varchar(45)	YES		<null>	
3	last_name	varchar(45)	YES		<null>	
4	email_id	varchar(45)	YES		<null>	

Figure 2 – Structure de la table employee (DESCRIBE)

La connexion est configurée dans le fichier **application.properties** : URL JDBC, identifiants et *ddl-auto=none*, car la table a été créée manuellement.



```
spring.application.name=demo
spring.datasource.url=jdbc:mysql://localhost:3306/employee-schema
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=none
```

Figure 3 – Fichier application.properties

5. Tests de l'API

Après le démarrage de l'application (port 8080), les endpoints ont été testés. Une requête **POST /api/employees** avec un corps JSON crée un employé :



```
###
POST http://localhost:8080/api/employees
Content-Type: application/json

{
  "firstName": "test",
  "lastName": "testing",
  "emailId": "test@teststing.com"
}

<> 2026-11-02T125606.200.json
```

Figure 4 – Requête POST /api/employees

Le serveur répond par un code **200** et renvoie l'employé enregistré avec son identifiant généré (id = 1) :



Figure 5 – Réponse de la requête POST

Une requête **GET /api/employees** retourne ensuite la liste des employés, dont celui qui vient d'être créé :

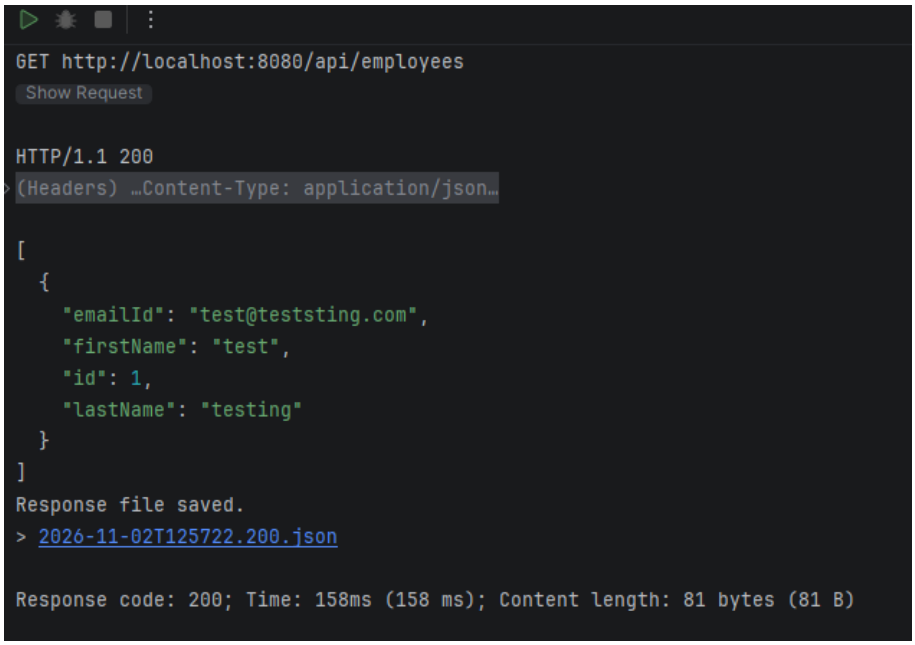


Figure 6 – Réponse de la requête GET

6. Conclusion

Ce TP a permis de mettre en place un service web REST complet avec Spring Boot : architecture en couches, persistance avec Spring Data JPA et MySQL, et exposition d'endpoints CRUD. Les tests POST et GET confirment que l'API communique correctement avec la base de données.